Technical Challenges and Solutions in Embedded Router Simulation

Ahmed Abdelmoula
220303923
Bilgisayar Müh/Comp. Eng.

January 23, 2026

Project Problems and Solutions

Below are the ten critical technical challenges encountered during the development of the Routing Simulator and the specific solutions implemented to address them.

1. **Linker Errors with Winsock Library**

   The Problem: The compiler threw "Unresolved External Symbol" errors (LNK2019) when using socket functions like `socket()`, `bind()`, or `accept()`.

   Solution: I realized that the standard C++ compiler does not link the Windows Socket library by default. I added the preprocessor directive `#pragma comment(lib, "ws2_32.lib")` to manually link the `Ws2_32.lib` library.

2. **Simulating Limited Memory (Embedded Constraint)**

   The Problem: The laptop has 16GB of RAM, but the project required simulating an embedded device with only 16MB. Using standard `std::vector` allowed unlimited growth, violating requirements.

   Solution: I implemented a fixed-size memory constraint. I defined a constant `TOTAL_SYSTEM_RAM = 16MB` and manually allocated a static array (`new char[]`) of 4MB for the buffer.

3. **Implementing Circular Buffer Logic**

The Problem: When writing data to the buffer, the `tail` pointer would eventually reach the end of the array, causing an "Index Out of Bounds" exception if simply incremented.

Solution: I implemented Modulo Arithmetic for the pointers. By using the formula `tail = (tail + 1) % capacity`, the pointer automatically wraps around to index 0, creating a true "Ring" structure.

4. **Handling TCP Data Fragmentation**

   The Problem: TCP is a stream protocol. Sometimes, `recv()` would return only part of the data (e.g., 1024 bytes instead of 4096), causing the image file to be corrupted.

   Solution: I implemented a Looping Receive Mechanism. The code loops and receives data chunks until the `totalBytesReceived` equals the expected `fileSize` from the header.

5. **Buffer Overflow Protection (Backpressure)**

   The Problem: The Sender was transmitting data faster than the Router could process. This risked overwriting unread data in the Circular Buffer.

   Solution: I added a check: `if (currentSize + length > capacity)`. If full, the Router stops receiving (blocking the socket) until space is freed up.

6. **Defining the End of File (EOF)**

   The Problem: The Receiver node did not know when the file transfer was complete, causing it to hang indefinitely.

   Solution: I designed a custom Protocol Header (`struct FileHeader`) containing the filename and exact file size. This allows the Receiver to know exactly how many bytes to expect.

7. **Corrupted Image Data (Binary vs. Text)**

   The Problem: Initially, the received `.jpg` image was unreadable because I was treating data as strings, which terminated early upon hitting a null terminator.

   Solution: I switched to Binary Mode for file I/O and `char` buffers. I ensured that `read()` and `write()` functions handled raw bytes without interpretation.

8. **Synchronization of Multiple Terminals**

   The Problem: If the Sender tried to connect before the Router was running, the connection failed with "Connection Refused."

   Solution: I established a strict Startup Sequence: 1) Start Router (Server), 2) Start Receiver, 3) Start Sender to initiate connection.

9. **Optimization of Routing Algorithm Overhead**

   The Problem: Running Dijkstra's algorithm for every single data packet caused unnecessary CPU overhead and latency.

   Solution: I optimized the logic to run the Routing Algorithm only once at the beginning of the session. The Router then establishes a virtual circuit for all subsequent packets.

10. **Pointer Arithmetic in Raw Memory**

    The Problem: Managing `head` and `tail` pointers manually was error-prone, leading to reading garbage data.

    Solution: I encapsulated the logic inside a `CircularBuffer` class and maintained a `currentSize` variable. The `read` function only executes if `currentSize > 0`.